

# Advanced Certificate Programme in Agentic AI & RAG Engineering

**FastAPI + Minimal UI + Testing Intro**

Phase 1 · Foundations, Eval Literacy & System Design



# Today's outline

Day 1 · Wrap it

## Concepts

### Three building blocks

**FastAPI Patterns** 20 min

**Streaming via StreamingResponse**  
15 min

**Minimal UI · Streamlit** 15 min

## Live demos

### Watch it built

**Build /ask + /health** 25 min

**Add StreamingResponse** 15 min

**Wire Streamlit to /ask** 15 min

## Lab Step 1 starts in class

### Build yours

`src/api/main.py`

`src/ui/app.py`

Streaming /ask end-to-end

Smoke test with capstone Qs

# You have a pipeline. Today it gets a surface.

Where we are

By end of W2

## An async batch pipeline

Reads questions from CSV  
Calls the LLM in parallel batches  
Retries on transient failures  
Logs JSON; persists to SQLite  
*Callable from Python only*



By end of W3

## An HTTP service

`/ask` — POST a question, get a streamed answer  
`/health` — for monitoring  
Minimal UI — any user can try it  
A few mocked unit tests  
Locked API contract in the ADR

# FastAPI Patterns

---

App, routes, Pydantic, validation — the minimum that matters.

# FastAPI Patterns

## Why wrap the pipeline as an API?

1

### Different surfaces, same brain

A UI, a Slack bot, a scheduled job, a notebook — none of them want to import your Python module. They all want to hit /ask.

2

### Real consumers

Your stakeholders won't run ``python pipeline.py``. The API is what makes the capstone usable by anyone who isn't you.

3

### A stable boundary

The API contract you lock today is what W4 will preserve while it upgrades everything inside. The contract is the bridge.

# FastAPI Patterns

The minimum FastAPI app — 12 lines



```
# api/main.py
from fastapi import FastAPI
from pipeline.models import Question, Answer
from pipeline import ask_llm

app = FastAPI(title="Capstone API")

@app.post("/ask", response_model=Answer)
async def ask(q: Question) -> Answer:
    return await ask_llm(q.question)

@app.get("/health")
async def health():
    return {"status": "ok"}
```

## Reuse Question + Answer

from W2 — no new models

## @app.post + Pydantic =

free validation + docs

## /health for monitoring —

you'll thank yourself in W29

# FastAPI Patterns

## Pydantic carries the contract



```
# pipeline/models.py (from W2 – no changes)
from pydantic import BaseModel, Field

class Question(BaseModel):
    question: str = Field(..., min_length=1)

class Answer(BaseModel):
    content: str
    cost_usd: float = 0.0001
    retries: int = 0
```

### What this buys you

#### ✓ Request validation

Empty question → 422 automatically

#### ✓ Response shape guaranteed

Field missing → server-side error,  
not a confused client

#### ✓ Free OpenAPI docs at /docs

Stakeholders try the API in a browser

# FastAPI Patterns

## /health — boring, essential

A two-line endpoint that returns `{"status": "ok"}`. That's it.

### Liveness probes

Kubernetes + most container platforms ping `/health` to know whether to restart your service.

### Smoke tests

Hit it in CI. If `/health` fails, deploys halt.

### Dependency check

Later you'll extend it to verify the database, LLM credentials, and config. Today, just say ok.

# FastAPI Patterns

## Run It



```
$ uvicorn api.main:app --reload --port 8000
INFO:      Will watch for changes in these directories: ['/home/learner/capstone']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [12453] using StatReload
INFO:      Started server process [12455]
INFO:      Application startup complete.

$ curl -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "What is leave policy?"}'
{"content": "Employees get 24 days of paid leave...", "cost_usd": 0.0001, "retries": 0}

$ curl http://localhost:8000/health
{"status": "ok"}
```

# Streaming

---

First token under one second. Users feel the difference.

# Streaming

Two response patterns. Same words. Different feel.

## Non-streaming

### Wait, then dump

0.0s click submit  
... user stares at a spinner  
5.4s whole answer appears

*Feels broken until it isn't.*

## Streaming

### Tokens arrive as the LLM types

0.0s click submit  
0.4s "Employees..."  
0.8s "... get 24 days..."  
5.4s final period

*Feels alive from token one.*

# Streaming

## StreamingResponse + async generator

```
from fastapi.responses import StreamingResponse

async def stream_answer(question: str):
    answer = await ask_llm(question)
    for word in answer.content.split(" "):
        yield word + " "
        await asyncio.sleep(0.05) # simulate real stream

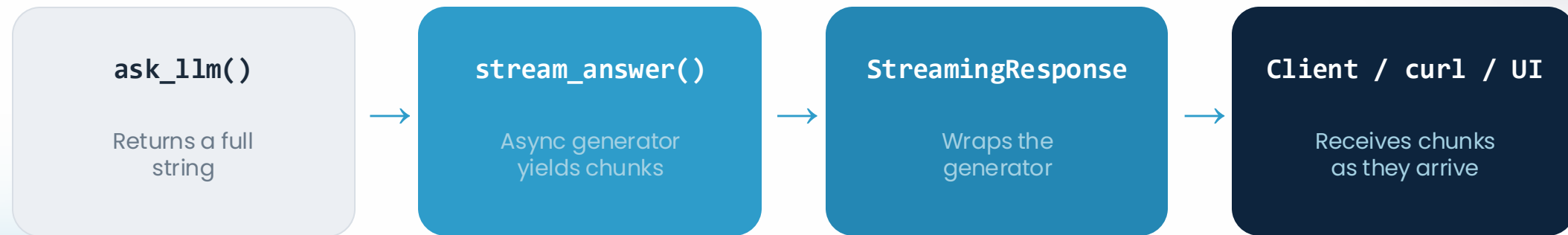
@app.post("/ask")
async def ask(q: Question):
    return StreamingResponse(
        stream_answer(q.question),
        media_type="text/plain",
    )
```

### Three rules

1. Use yield, not return  
Generator, not coroutine.
2. Wrap in StreamingResponse  
Or FastAPI buffers it.
3. Set media\_type  
Clients need to know.

# Streaming

Where the chunks come from



*This week we stream the response from FastAPI to the client.*

*W4 will stream from the LLM itself — end-to-end token streaming.*

# Streaming

Run It — chunks arriving



```
$ curl -N -X POST http://localhost:8000/ask \  
  -H "Content-Type: application/json" \  
  -d '{"question": "What is the leave policy?"}'
```

```
Employees  
get  
24  
days  
of  
paid  
leave  
per  
year.
```

```
# -N flag = no buffering – see them arrive
```

# Minimal UI • Streamlit

---

Ten lines and your API has a real face. Stakeholders can try it.

# Minimal UI

Streamlit — 10 lines, calls /ask, streams in

```
# ui/app.py
import streamlit as st
import requests

st.title("Capstone Q&A")
question = st.text_input("Ask anything")

if question:
    placeholder = st.empty()
    answer = ""
    with requests.post("http://localhost:8000/ask",
                      json={"question": question},
                      stream=True) as r:
        for chunk in r.iter_content(decode_unicode=True):
            answer += chunk
            placeholder.markdown(answer)
```

**Runs on Vocareum at port 8501**

`streamlit run ui/app.py`

`st.empty()` lets us redraw the same spot on each chunk — the answer appears to type itself.

`stream=True` on `requests.post()` is the streaming handshake.

# Minimal UI

Streamlit by default — Chainlit if you want chat-shaped

**STREAMLIT** · what we use

## General-purpose UI for Python

- ✓ Single port (8501), works on Vocareum
- ✓ Chat, dashboards, tables, charts, all in one
- ✓ Same UI scales up for W6+ retrieval views
- ✓ Closest to 'normal Python file you run'

**CHAINLIT** · optional alternative

## Built for chat UIs

- Pretty chat threads out of the box
- Streaming + history baked in
- Less general than Streamlit
- *Pick this only if your capstone is chat-shaped*

# Live demo recap

What you just watched me build

1

## FastAPI app

Wrapped W2's ask\_llm in /ask + added /health. Pydantic Question/Answer carried the contract. Hit /docs in the browser.

25 min

2

## Streaming /ask

Replaced the synchronous return with StreamingResponse + an async generator. curl -N showed chunks arriving every 50 ms.

15 min

3

## Streamlit UI

10-line app.py — text input → requests.post(stream=True) → st.empty().markdown updating on each chunk.

15 min

# In the Lab

---

Now you build it. 45 minutes. Starting in class, finishing tonight.

# Lab Step 1

Build `src/api/main.py` — and an honest Streamlit UI

## WHAT YOU'RE BUILDING

### A real HTTP service

- 1 FastAPI app**  
`/ask` + `/health`, Pydantic in front
- 2 Streaming `/ask`**  
StreamingResponse + async generator
- 3 Streamlit UI**  
`ui/app.py` hitting `/ask`
- 4 Smoke test**  
3 capstone questions end-to-end

## SUB-STEPS • IN THE LAB GUIDE

### Eight beats, seven-beat shape

- 1a** Project structure & deps
- 1b** Create `main.py` + FastAPI app
- 1c** Add `/ask` (non-streaming first)
- 1d** Add `/health`
- 1e** Run uvicorn + curl smoke
- 1f** Convert `/ask` to StreamingResponse
- 1g** Create `ui/app.py` (Streamlit)
- 1h** Wire UI to `/ask` end-to-end

# Lab Step 1 • outcome

What you should see by bed-time



```
# Terminal 1 – your API
$ uvicorn api.main:app --reload --port 8000
INFO:      Application startup complete.
INFO:      127.0.0.1 - "POST /ask HTTP/1.1" 200 OK

# Terminal 2 – your UI
$ streamlit run ui/app.py --server.port 8501
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501

# Browser at 8501 – you ask a real capstone question:
"How does the procurement approval process work?"
# And see tokens appear one-by-one until the full answer fills in. ←
```

# Sandbox

Start Step 1 — we're here while you do

Right now, on your machine:

**Open the lab guide. Aim for sub-steps 1a → 1d.**

By end of class you should have `/ask` + `/health` responding to curl.

- ↳ Stuck on imports? Pair with someone next to you. Walk to the front for an instructor.
- ↳ Finish 1e → 1h tonight. Day 2 starts with everyone running an API.

# Day 1 • Wrap

You saw three pieces fit together

## FastAPI

Wrapped W2's pipeline as `/ask` + `/health`. Pydantic carries the contract.

## Streaming

`StreamingResponse` + async generator.  
Tokens feel alive.

## Streamlit UI

Ten lines + `stream=True`. Anyone can try the capstone.

## TOMORROW • DAY 2

Testing discipline + API contracts. Write tests, stress the service, lock the contract.

# Day 2 • Polish it

## Today's outline

### Concepts

#### Two disciplines

##### Testing 25 min

Deterministic code behaviour,  
mocks, pytest.

##### API Contracts 20 min

/v1/ask is locked from today.

### Live demos

#### Two short builds

##### Mocked unit test 20 min

One test, the right shape.

##### Write the contract 15 min

/v1/ask + versioning rule, into ADR.

### Lab tonight

#### Three steps

##### Step 2: Mocked tests 30 min

##### Step 3: Stress-test 45 min

##### Step 4: ADR contract 15 min

# Quick recap

Show of hands — by last night you had...

- ✓ **/ask returning a streamed answer**  
*via curl*
- ✓ **/health returning ok**
- ✓ **Streamlit UI running on 8501**  
*tokens streaming in*
- ✓ **Smoke test passing on 3 capstone questions**

If any of these isn't checked

## Speak up now

We'll pair you with a learner who finished. Today's content assumes a running service to test.

# Testing

---

Deterministic behaviour of your code. The LLM is mocked.

# Testing

Two disciplines. Don't conflate them.

## TESTING · THIS WEEK

### Code behaviour, deterministic

**Question:** "Does my code call the right things in the right order?"

**Method:** Mock the LLM. Assert your retry triggers. Assert /ask rejects malformed input with 422.

*Same input → same answer, every time.*

## EVALUATION · W5

### LLM output quality, probabilistic

**Question:** "Was that answer any good?"

**Method:** Real LLM call. Golden set. LLM-as-judge against a rubric. Pairwise comparison.

*Different tools, different question.*

# Testing

## pytest — minimum viable shape

```
# tests/test_pipeline.py
import pytest
from unittest.mock import AsyncMock, patch
from pipeline import ask_llm

@pytest.mark.asyncio
async def test_ask_llm_calls_client_once():
    # Arrange
    mock_client = AsyncMock()
    mock_client.complete.return_value = "Hi"
    # Act
    with patch("pipeline.client", mock_client):
        result = await ask_llm("What is RAG?")
    # Assert
    assert mock_client.complete.call_count == 1
```

### The shape

#### Arrange

Set up mocks + fake returns.

#### Act

Call the function under test.

#### Assert

Check behaviour — not output quality. That's W5's job.

# Testing

Mock the boundary you don't control

BOUNDARY

YOUR CODE

`ask_llm()`

Retry logic, validation, error paths. Test this.

OPENAI API

`AsyncOpenAI()`

Network, money, non-determinism. Mock this.

RULE

**Test what you wrote.  
Mock what you  
imported.**

# Testing

Three tests worth writing this week

01

## **ask\_llm calls client correctly**

Mock the OpenAI client. Call `ask_llm` with a question. Assert the client got called once with the right model + the right messages.

***Catches: prompt drift, model rename, wrong API.***

02

## **Retry hits 3 times on 500**

Mock the client to raise `httpx.HTTPStatusError(500)`. Call `ask_llm_with_retry`. Assert the mock was called 3 times before raising.

***Catches: silent retry breakage.***

03

## **/ask rejects malformed input with 422**

Use `TestClient`. POST with missing 'question' field. Assert `response.status_code == 422`.

***Catches: Pydantic validation regressions.***

# Testing

Run It — pytest green

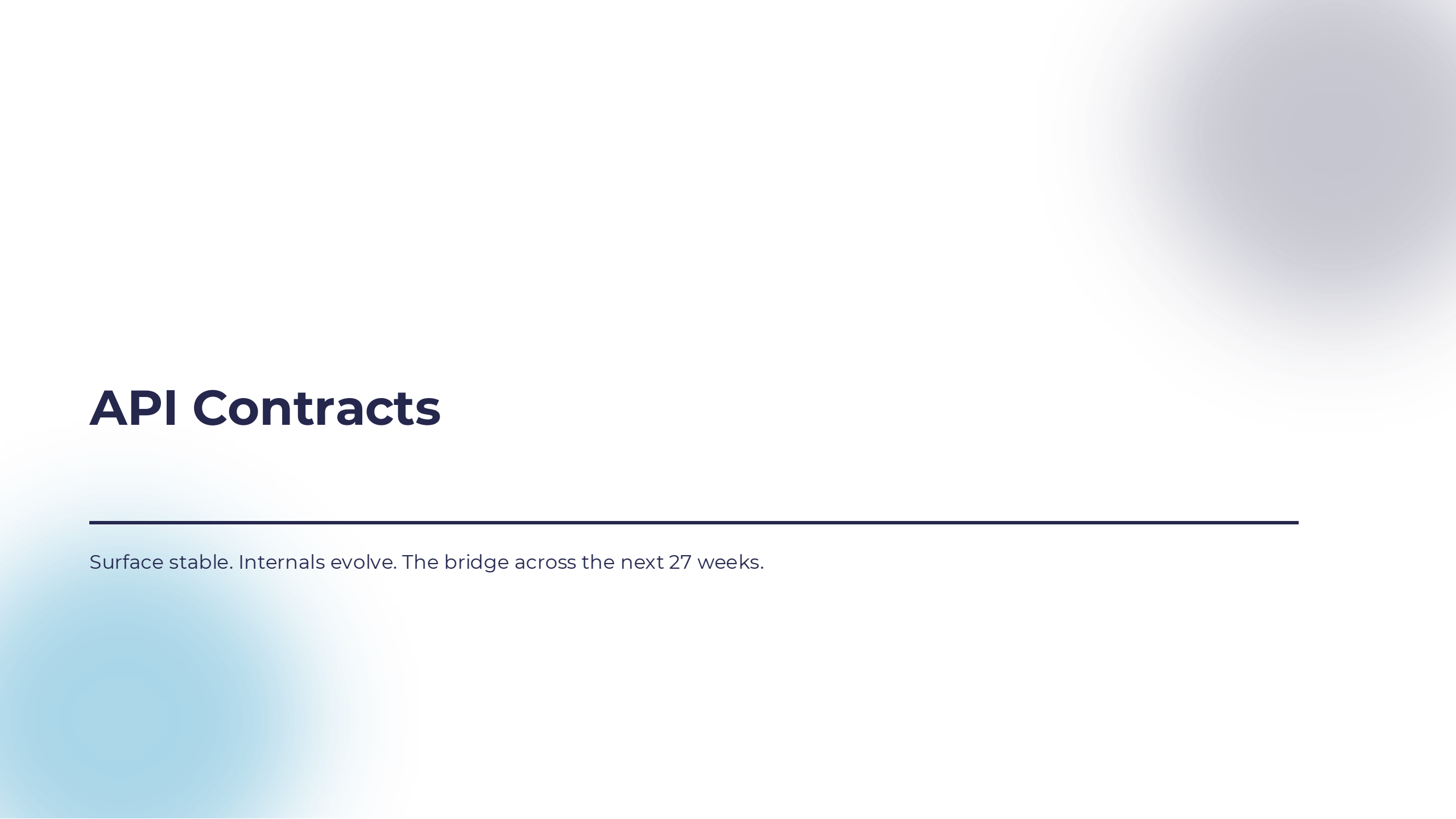


```
$ pytest tests/ -v
===== test session starts =====
platform linux -- Python 3.11.5, pytest-7.4.0, pluggy-1.3.0
plugins: asyncio-0.21.1
collected 3 items

tests/test_pipeline.py::test_ask_llm_calls_client_once PASSED [ 33%]
tests/test_pipeline.py::test_retry_three_times_on_500 PASSED [ 66%]
tests/test_api.py::test_ask_rejects_missing_question PASSED [100%]

===== 3 passed in 0.42s =====
                                     < 0.42s total
                                     < no real API calls
```

# API Contracts



---

Surface stable. Internals evolve. The bridge across the next 27 weeks.

# API Contracts

You already wrote a contract — yesterday

Every time you defined a Pydantic model in front of an endpoint:



```
# This is your /v1/ask contract:
```

```
class Question(BaseModel):  
    question: str                # REQUEST →
```

```
class Answer(BaseModel):  
    content: str                 # RESPONSE ←  
    cost_usd: float  
    retries: int
```

**That's the contract.**

Callers commit to sending Question.

You commit to returning Answer.

**Locked from today.**

**Versioned from today.**

# API Contracts

When do you bump from v1 to v2?

## DON'T BUMP — keep /v1/ask

- Adding a new optional field to Answer
- Improving the LLM prompt internally
- Swapping models (GPT → Claude)
- Adding streaming behind the same endpoint
- Fixing bugs in retry logic
- Adding telemetry / logging

*Callers shouldn't notice. Often, they don't.*

## BUMP TO /v2/ask — breaking change

- Removing a field from Answer
- Renaming a field (content → text)
- Changing a field's type (str → list)
- Making an optional field required
- Changing error response shape
- Changing request semantics

*Old callers will break. They need warning + time.*

# API Contracts

## The W4 promise

***"Next week, everything inside /ask changes. The contract holds."***

### W4 changes the internals

- Token + cost tracking from response.usage
- Tool-calling for structured outputs
- Streaming from the LLM, not just from FastAPI

### W4 doesn't touch

- The /v1/ask URL
- Pydantic Question (request shape)
- Pydantic Answer field names

# API Contracts

Write it into the ADR



```
# docs/adr/0002-api-contract.md
```

```
## /v1/ask
```

```
**Request:** Question(question: str)
```

```
**Response:** Answer(content: str, cost_usd: float, retries: int)
```

```
**Behaviour:** Streamed text/plain. Same content available batched at /v1/ask?stream=false.
```

```
## Versioning rule
```

- /v1 is locked from W3 onward.
- Add fields freely (callers ignore unknowns).
- Bump to /v2 only on breaking shape change. Old /v1 runs in parallel for  $\geq 2$  weeks.
- Schema versioning lands in W4 (schema\_version field on Answer).

# In the Lab

---

Three lab steps. ~90 minutes. Make your API hard to break.

# Lab Steps 2 · 3 · 4

Test it, stress it, lock the contract

## WHAT YOU'RE DOING

### Three steps, three deliverables

Step 2 30 min

#### Mocked unit tests

2-3 tests, pytest green

Step 3 45 min

#### Stress-test the API

4 attacks, document findings

Step 4 15 min

#### Lock the contract

ADR /v1/ask + versioning rule

## SUB-STEPS · TWELVE BEATS

- 2a tests/ + conftest
- 2b First mocked test (client args)
- 2c Retry test (3 hits on 500)
- 2d pytest green
- 3a Malformed JSON test
- 3b 5000-char question (latency)
- 3c Disconnect mid-stream
- 3d 50 parallel calls
- 3e Document findings
- 4a Open ADR
- 4b Write v1 contract section
- 4c Commit

# Lab outcome

## What you should see



```
$ pytest tests/ -v
collected 3 items
tests/test_pipeline.py::test_ask_llm_calls_client_once PASSED
tests/test_pipeline.py::test_retry_three_times_on_500 PASSED
tests/test_api.py::test_ask_rejects_missing_question PASSED
3 passed in 0.42s
```

```
$ cat docs/wk3-stress-notes.md
## Findings
- Malformed JSON: 422 with clear error ✓
- 5000-char Q: 4.8s total, streamed; OK ✓
- Disconnect mid-stream: server logs cleanly, no zombie task ✓
- 50 parallel: all 50 in SQLite, no rate-limit errors ✓
```

```
$ git log --oneline -3
a3f9c2b Lock /v1/ask contract in ADR 0002 ← you
```

# Sandbox

Start Step 2 — we're here while you do

Right now:

**Open the lab. Aim for sub-steps 2a → 2d in class.**

By end of class you should have pytest running green on at least one mocked test.

- ↳ AsyncMock confusion is normal — call us over. Better to fix it now than spiral tonight.
- ↳ Finish Steps 3 and 4 tonight. ADR contract written = W3 done.

# Week 3 • Wrap

You have a real API now

## W3 — SHIPPED

- ✓ **FastAPI service: /ask + /health**
- ✓ **Streaming via StreamingResponse**
- ✓ **Streamlit UI hitting /ask**
- ✓ **2–3 mocked unit tests, pytest green**
- ✓ **Stress-test findings in docs/**
- ✓ **ADR 0002 — /v1/ask contract locked**

## W4 — NEXT WEEK

### LLM Foundations + Model Landscape

*Open up the body of /ask:*

- Token + cost mechanics
- Three structured-output approaches
- Real LLM streaming (end-to-end)
- Model landscape: GPT/Claude/Ollama

*The /v1/ask contract holds.*

## Q & A

**Thank you**